

Reconocimiento Imágenes en Python

Luis Fernando Castillo
@luisfercastillo

Agenda

- Introducción aprendizaje
- Librerías
- Conceptos redes neuronales convolucionales
- Conceptos de Filtros
- Código en Python
 - Estructura
 - Entrenamiento
 - Pruebas

Supervisado

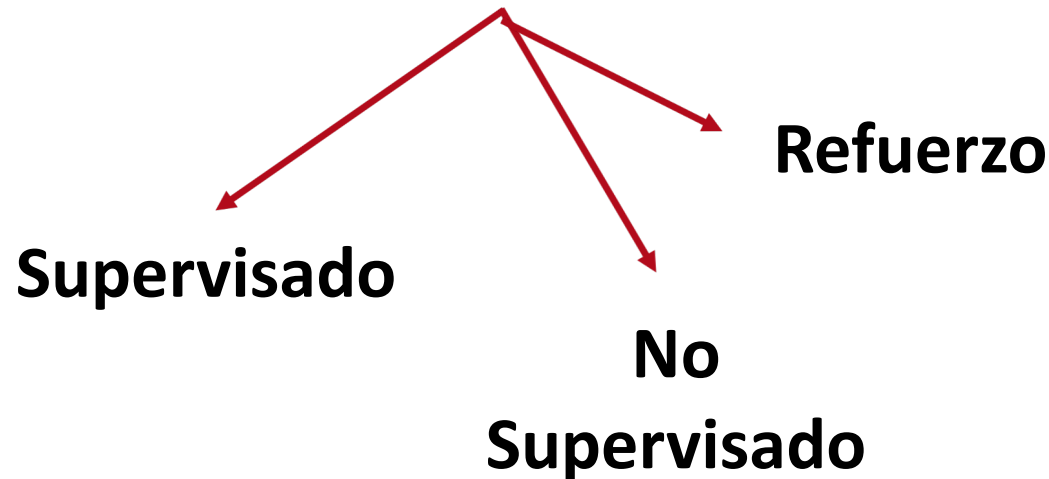
- ¡ *En el entrenamiento supervisado, se proporcionan tanto las entradas como las salidas. La red procesa las entradas y compara las salidas resultantes con las salidas deseadas.*

Utoronto, Source,

<http://www.psych.utoronto.ca/users/reingold/courses/ai/cache/neural3.html>

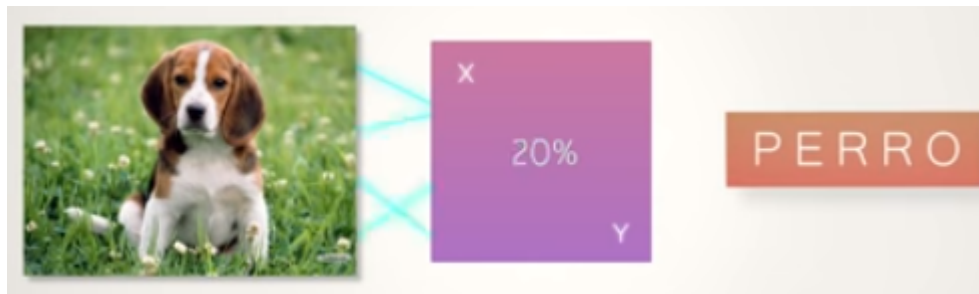
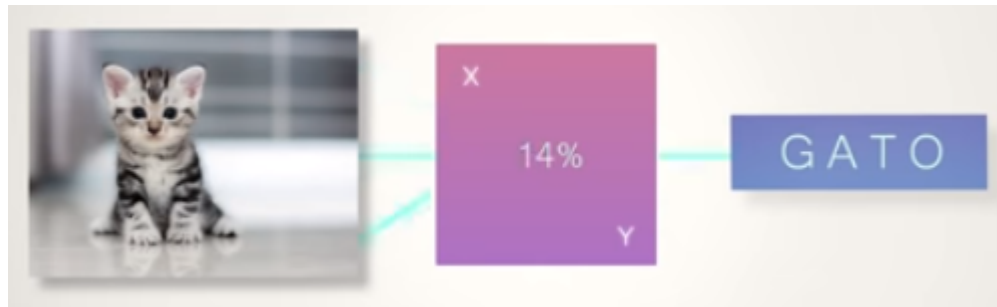
Tipos de Aprendizaje

Tipos de Aprendizaje en Inteligencia Artificial



Supervisado

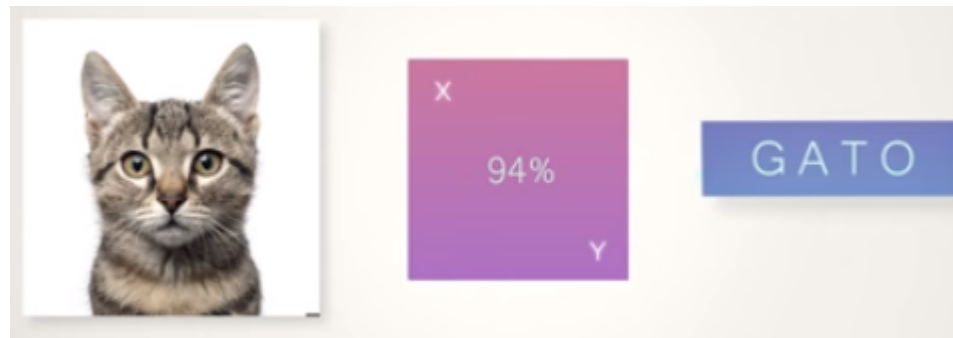
Ej: Detección mascotas



Dot CSV. Source <https://www.youtube.com/watch?v=oT3arRRB2Cw>

Supervisado

Ej: Detección mascotas



Dot CSV. Source <https://www.youtube.com/watch?v=oT3arRRB2Cw>

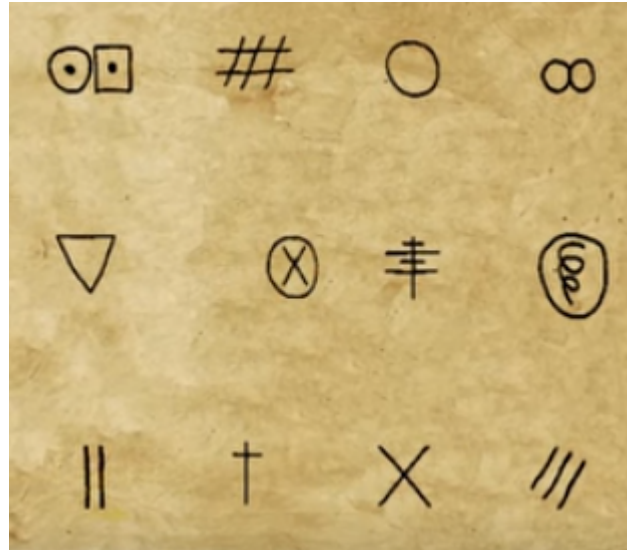
NO Supervisado

- ¡ *En el entrenamiento no supervisado, la red cuenta con entradas pero no con las salidas deseadas. El propio sistema debe decidir qué funciones utilizará para agrupar los datos de entrada.*

Utoronto, Source,

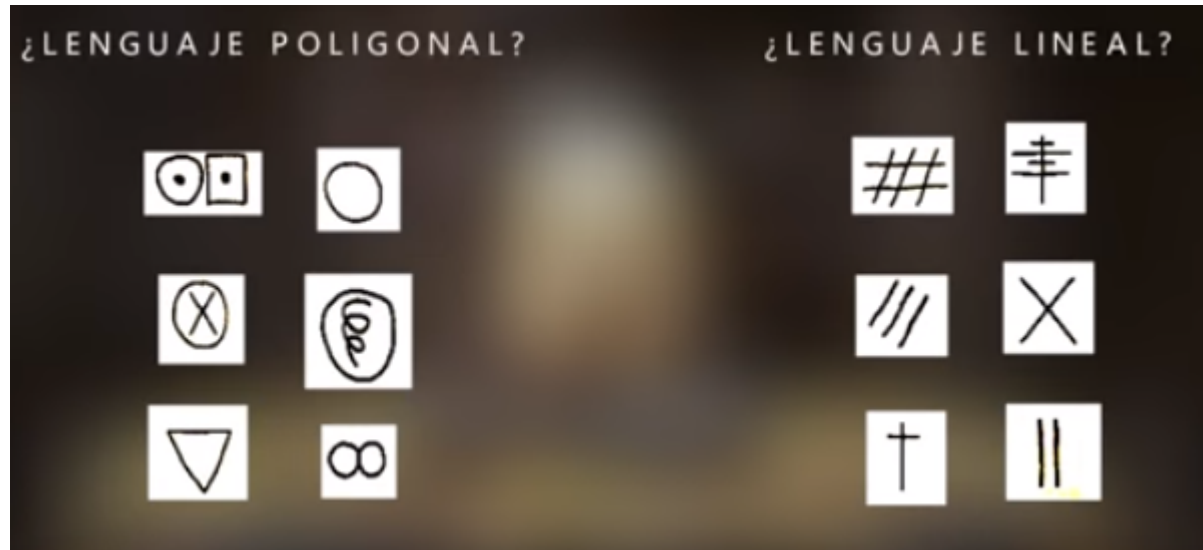
<http://www.psych.utoronto.ca/users/reingold/courses/ai/cache/neural3.html>

NO Supervisado



Dot CSV. Source <https://www.youtube.com/watch?v=oT3arRRB2Cw>

NO Supervisado



Dot CSV. Source <https://www.youtube.com/watch?v=oT3arRRB2Cw>

Supervisado



Observación

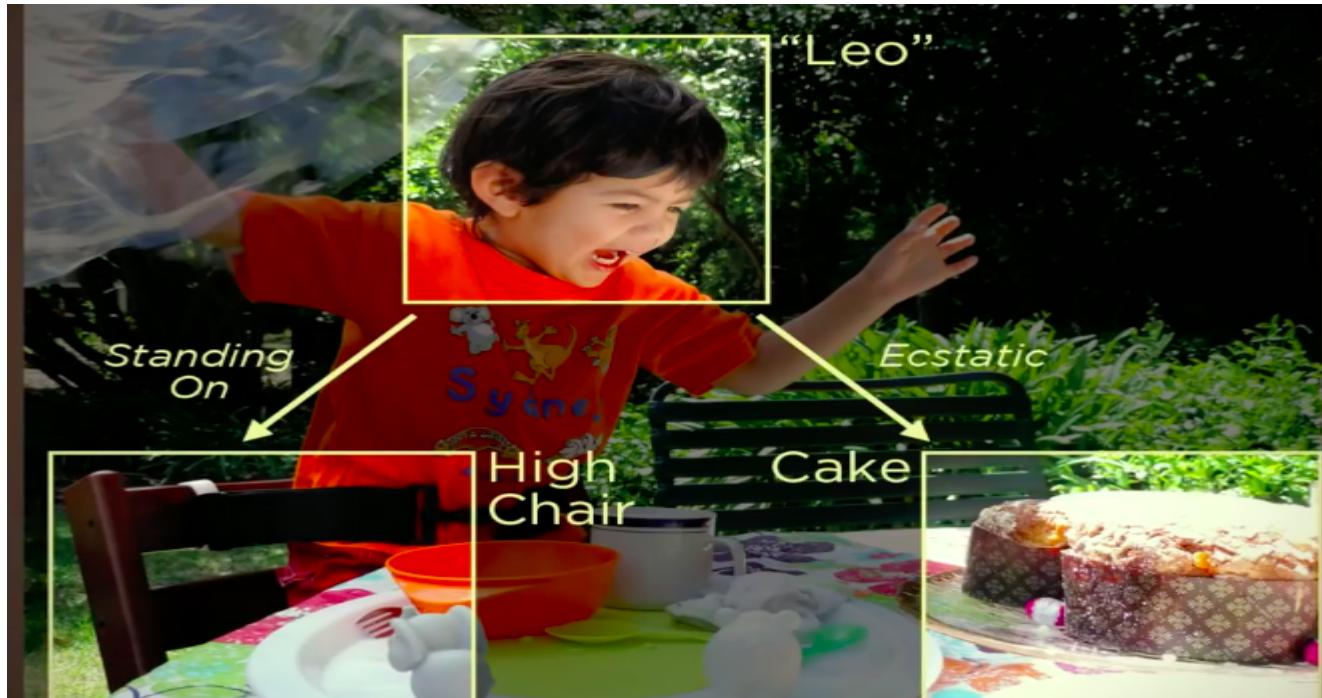
Observación



Observación

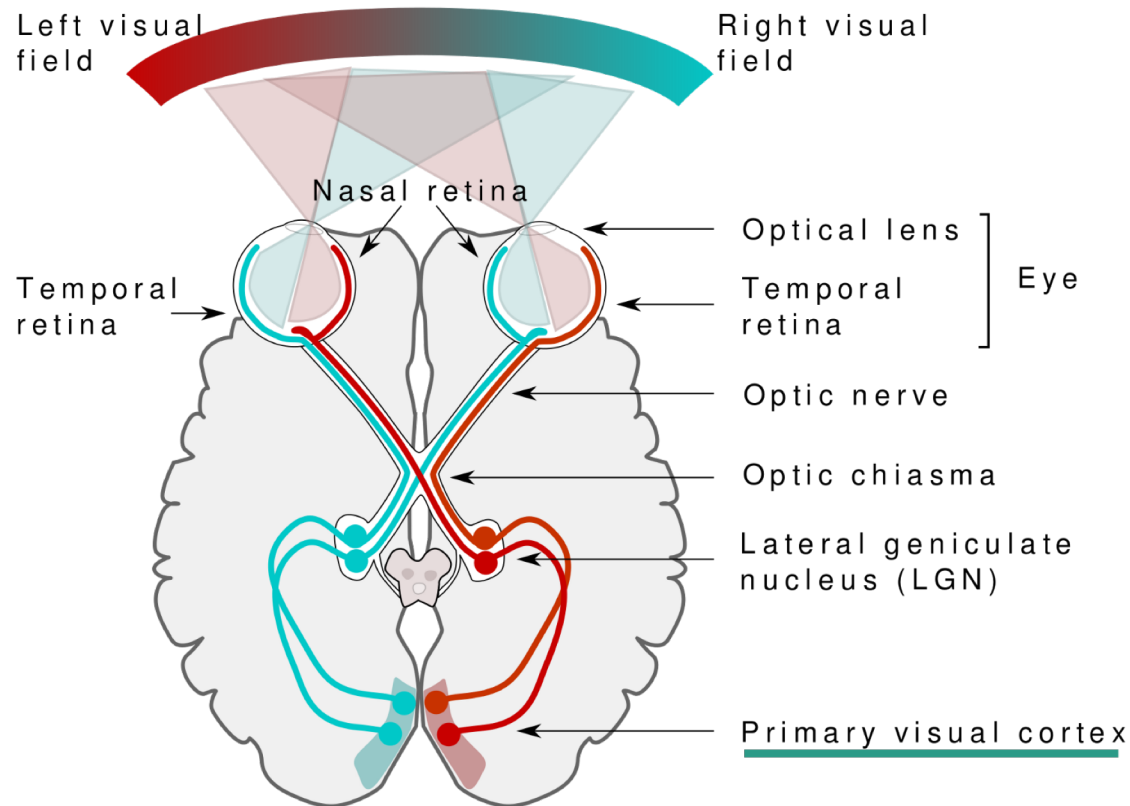
Source: <https://www.youtube.com/watch?v=40riCqvRoMs>

Observación



Source: <https://www.youtube.com/watch?v=40riCqvRoMs>

Procesamiento imágenes Cerebro



The Visual

Pathway. Source: https://commons.wikimedia.org/wiki/File:Human_visual_pathway.svg

Ejemplo Procesamiento

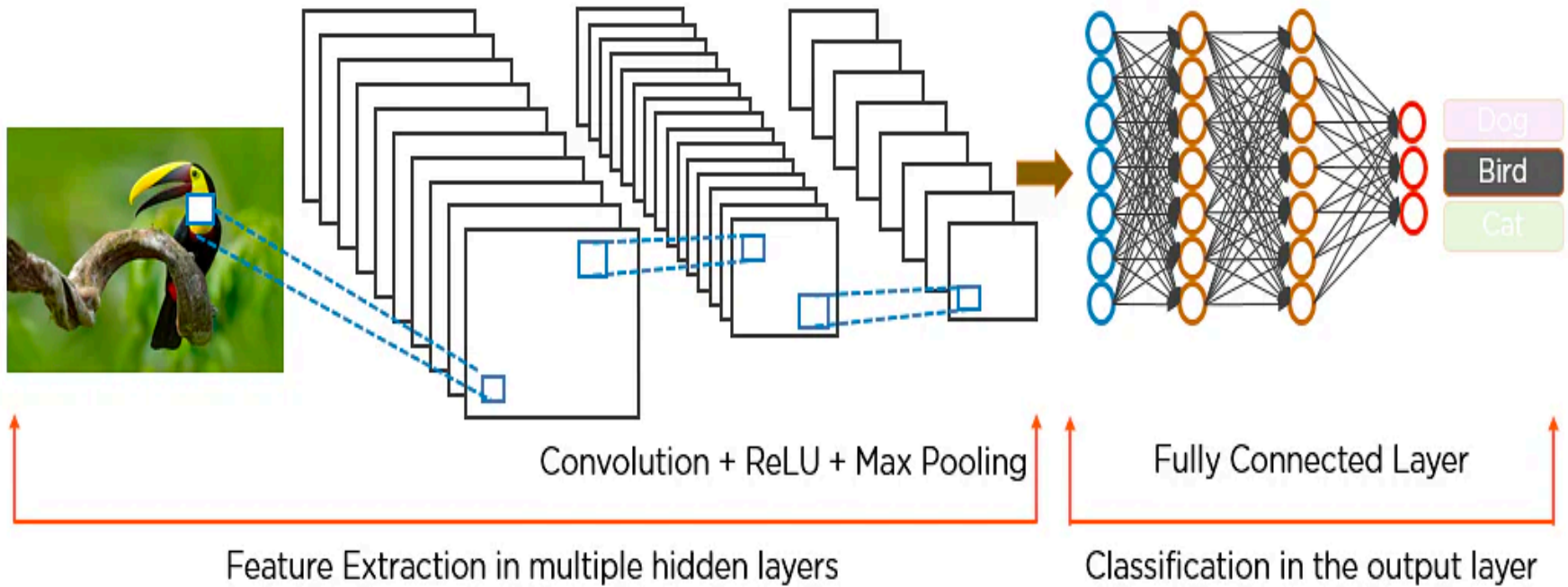


Entrenamiento



El cerebro ha tardado 500 millones de años para desarrollar esta habilidad

Red neuronal convolucional

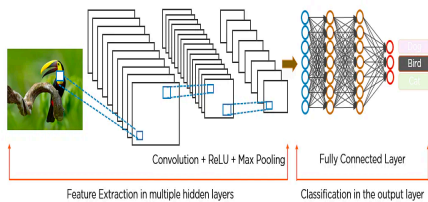


https://www.simplilearn.com/ice9/free_resources_article_thumb/CNN_recognizes_a_bird1.png

(Convolutional Neural Network CNN)

Red neuronal convolucional

Una red neuronal convolucional (Convolutional Neural Network CNN) es una clase de redes neuronales artificiales que utiliza capas convolucionales para filtrar entradas y obtener información útil.



Tiene diversas aplicaciones tales como:

- Reconocimiento y clasificación de imágenes,
- Análisis de texto y video
- Reconocimiento de voz
- Clasificación de texto entre otros.

(NVIDIA DEVELOPER,2018)

(Convolutional Neural Network CNN)



TensorFlow es una biblioteca de código abierto para aprendizaje automático a través de un rango de tareas, y desarrollado por Google para satisfacer sus necesidades de sistemas capaces de construir y entrenar redes neuronales para detectar y descifrar patrones y correlaciones, análogos al aprendizaje y razonamiento usados por los humanos.



Keras

Keras es una biblioteca de Redes Neuronales de Código Abierto escrita en Python. Es capaz de ejecutarse sobre TensorFlow, Microsoft Cognitive Toolkit o Theano.¹.

Está especialmente diseñada para posibilitar la experimentación en más o menos poco tiempo con redes de Aprendizaje Profundo. Sus fuertes se centran en ser amigable para el usuario, modular y extensible.

.



OpenCV es una biblioteca libre de visión artificial originalmente desarrollada por Intel. Open CV es multiplataforma, existiendo versiones para GNU/Linux, Mac OS X, Windows y Android. Contiene más de 500 funciones que abarcan una gran gama de áreas en el proceso de visión, como reconocimiento de objetos reconocimiento facial, calibración de cámaras, visión robótica entre otras.



OpenCV

4.6.0



TensorFlow

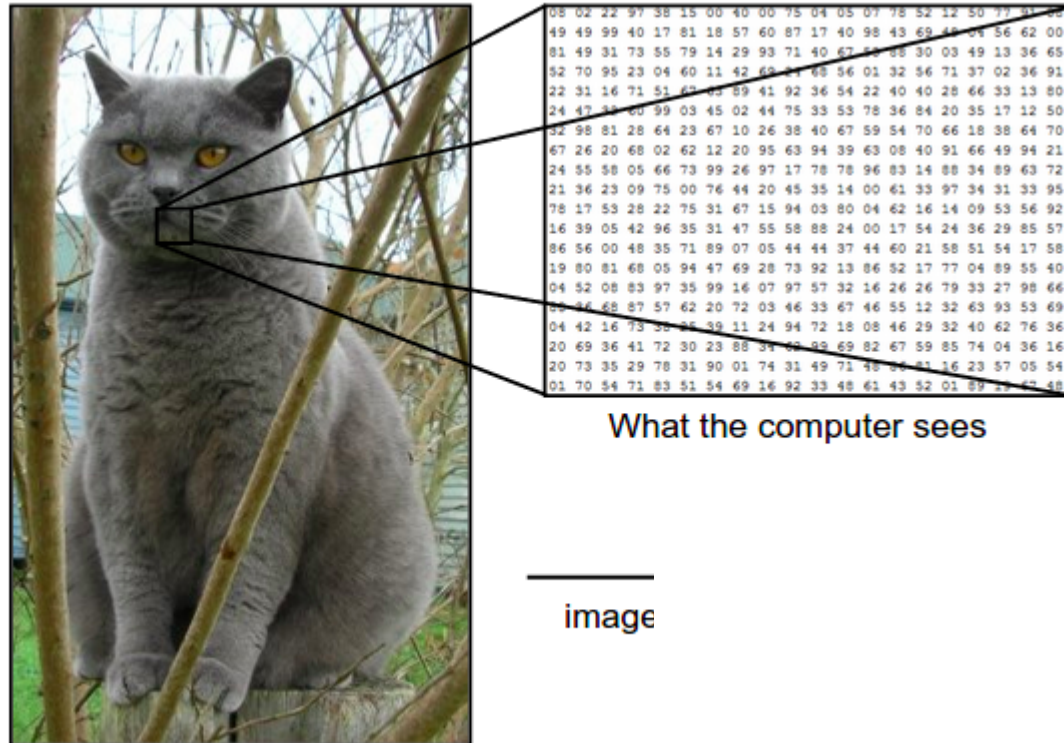
1.5.1



Keras

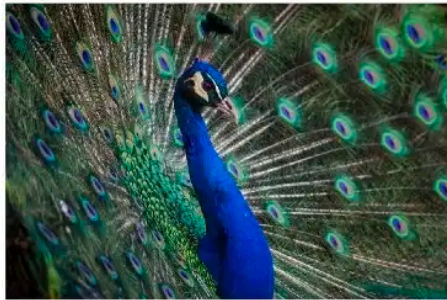
2.1.3

Detrás de las imágenes



Google Developers, <https://developers.google.com/machine-learning/practica/image-classification/convolutional-neural-networks>

Detrás de las imágenes



Red



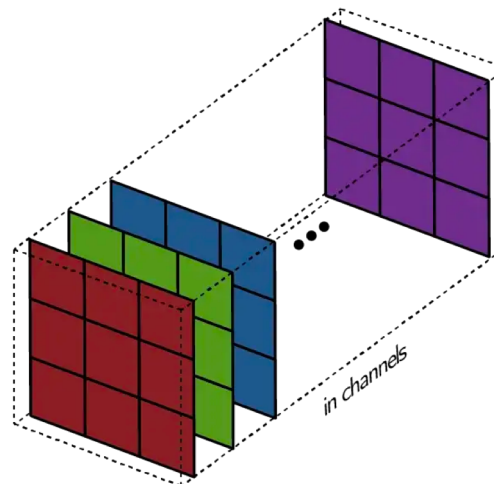
Green



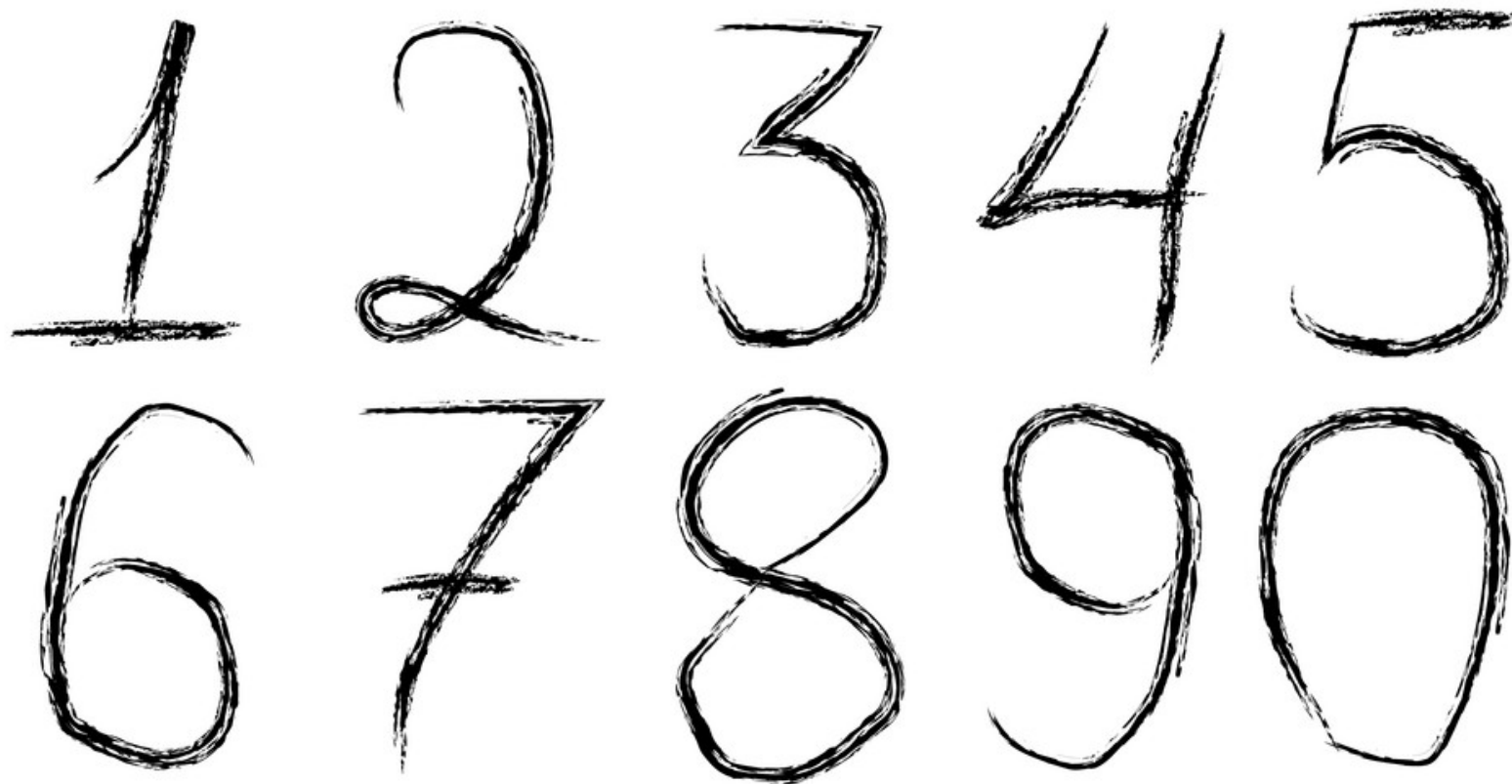
Blue



Most of the time, we deal with RGB images with three channels. (Credit: [Andre Mouton](#))



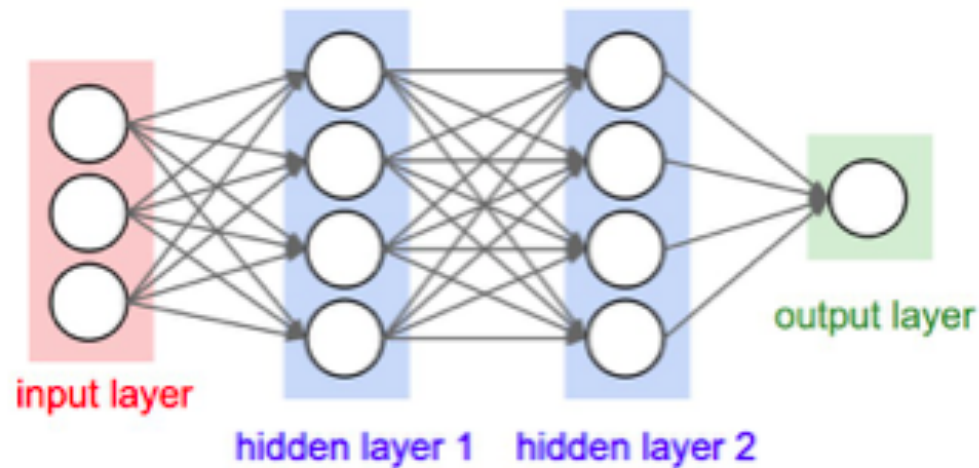
Detrás de las imágenes



Detrás de las imágenes

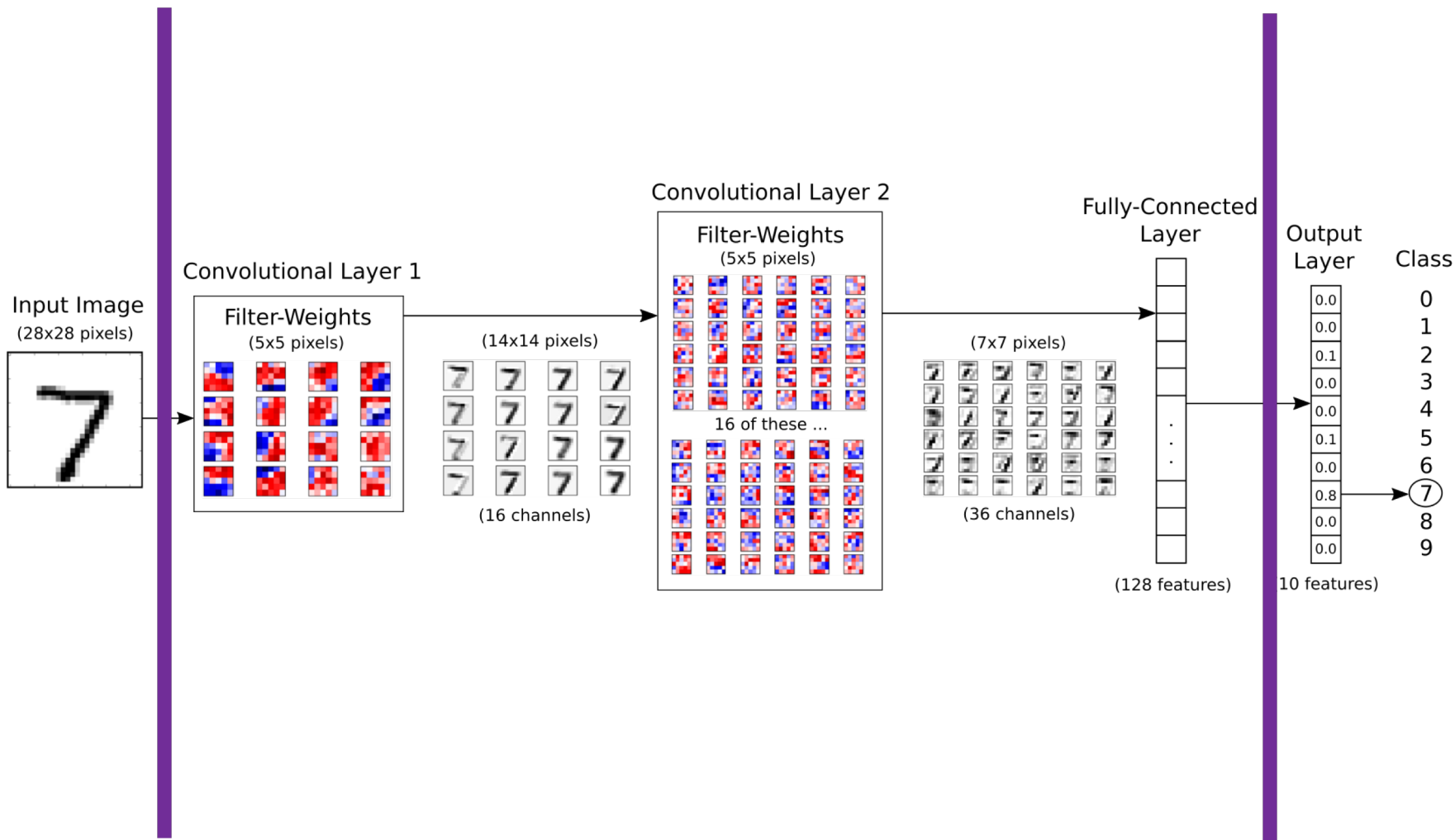


Estructura general Red Neuronal Artificial



Google Developers, <https://developers.google.com/machine-learning/practica/image-classification/convolutional-neural-networks>

Estructura general Red Neuronal Artificial



Detrás de las imágenes



What the computer sees

Código en Python

```
img_size=28
#Numero de neuronas de la cnn
img_size_flat=img_size*img_size
#Parametrizar la forma de imagenes
num_chanel=1
#RGB, HSV -> num_chanel=3
img_shape=(img_size,img_size,num_chanel)
num_clases=10
limiteImagenesPrueba=60
imagenes,etiquetas,probabilidades=cargarDatos("dataset/",num_clases,limiteImagenesPrueba)
```

Parámetros iniciales imagen

Código en Python

```
def cargarDatos(fase, numeroCategorias, limite):  
    imagenesCargadas=[]  
    etiquetas=[]  
    valorEsperado=[]  
    for categoria in range(0, numeroCategorias):  
        for idImagen in range(0, limite):  
            ruta=fase+str(categoria)+"/"+str(categoria)+"_"+str(idImagen)+".jpg"  
            print(ruta)  
            imagen=cv2.imread(ruta, 0)  
            imagen=imagen.flatten()  
            imagen=imagen/255  
            imagenesCargadas.append(imagen)  
            etiquetas.append(categoria)  
            probabilidades=np.zeros(numeroCategorias)  
            probabilidades[categoria]=1  
        valorEsperado.append(probabilidades)  
    imagenesEntrenamiento=np.array(imagenesCargadas)  
    etiquetasEntrenamiento=np.array(etiquetas)  
    valoresEsperados=np.array(valorEsperado)  
    return imagenesEntrenamiento, etiquetasEntrenamiento, valoresEsperados
```

Cargando dataset

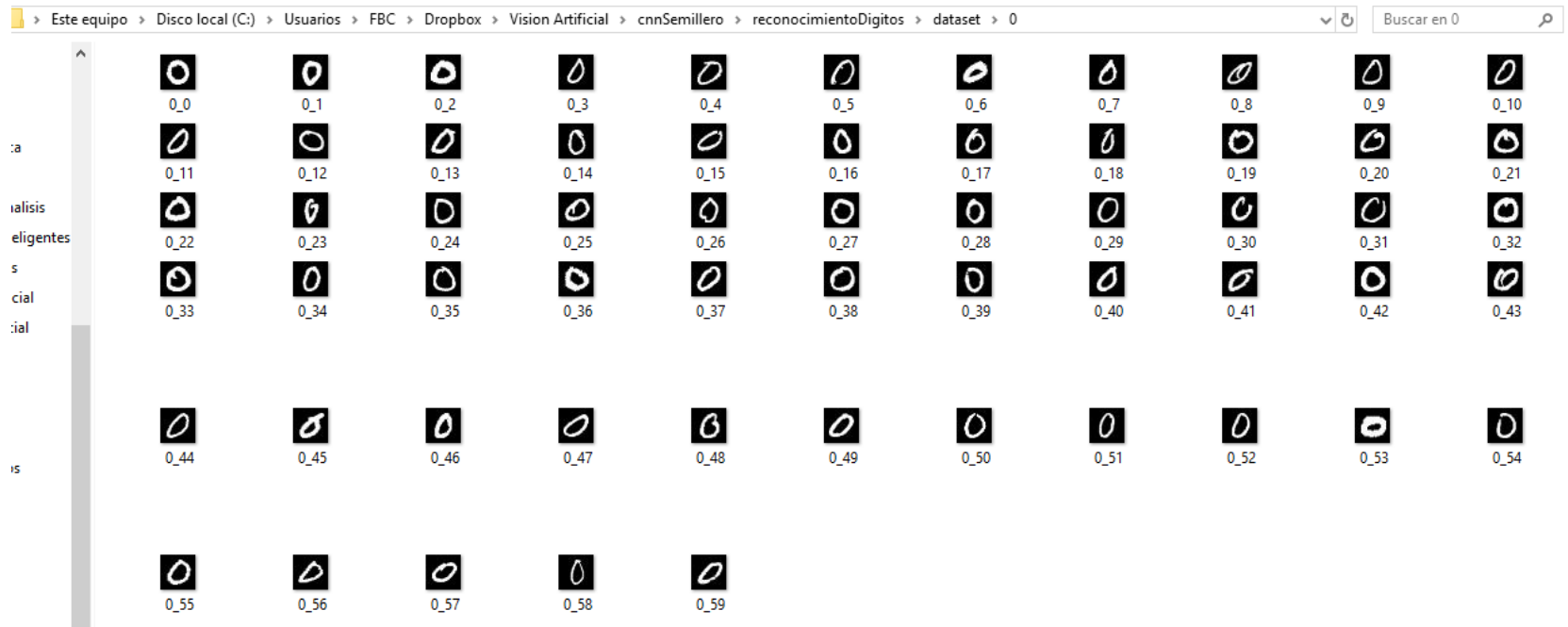
Código en Python

Este equipo > Disco local (C:) > Usuarios > FBC > Dropbox > Vision Artificial > cnnSemillero > reconocimientoDigitos > dataset

	Nombre	Fecha de modifica...	Tipo	Tamaño
	0	26/03/2019 9:52 a....	Carpeta de archivos	
es	1	26/03/2019 9:53 a....	Carpeta de archivos	
eca	2	26/03/2019 9:53 a....	Carpeta de archivos	
	3	26/03/2019 9:54 a....	Carpeta de archivos	
analisis	4	26/03/2019 9:54 a....	Carpeta de archivos	
nteligentes	5	26/03/2019 9:55 a....	Carpeta de archivos	
jos	6	26/03/2019 9:55 a....	Carpeta de archivos	
ifical	7	26/03/2019 9:55 a....	Carpeta de archivos	
	8	26/03/2019 9:56 a....	Carpeta de archivos	
ficial	9	26/03/2019 9:56 a....	Carpeta de archivos	

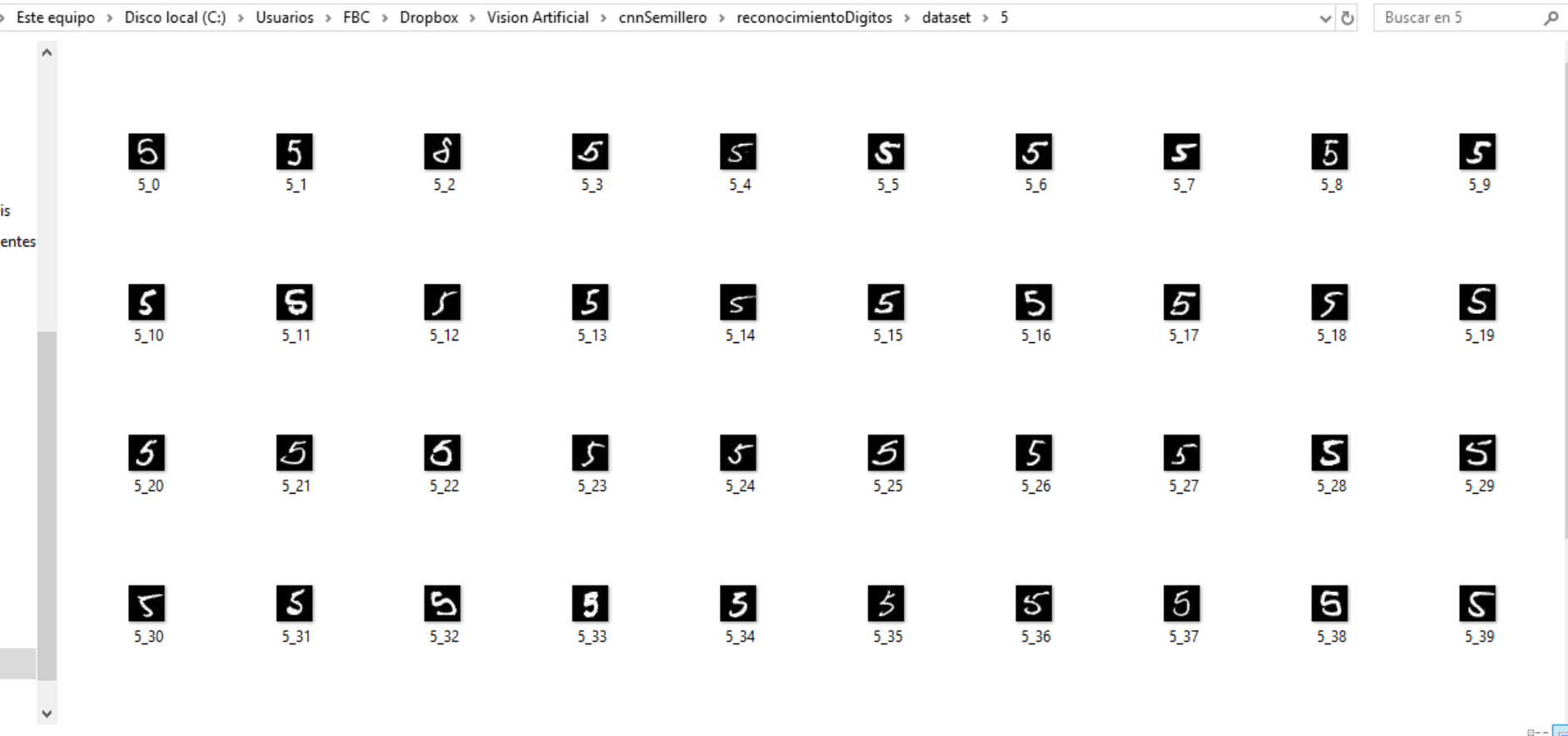
Dataset prueba

Dataset



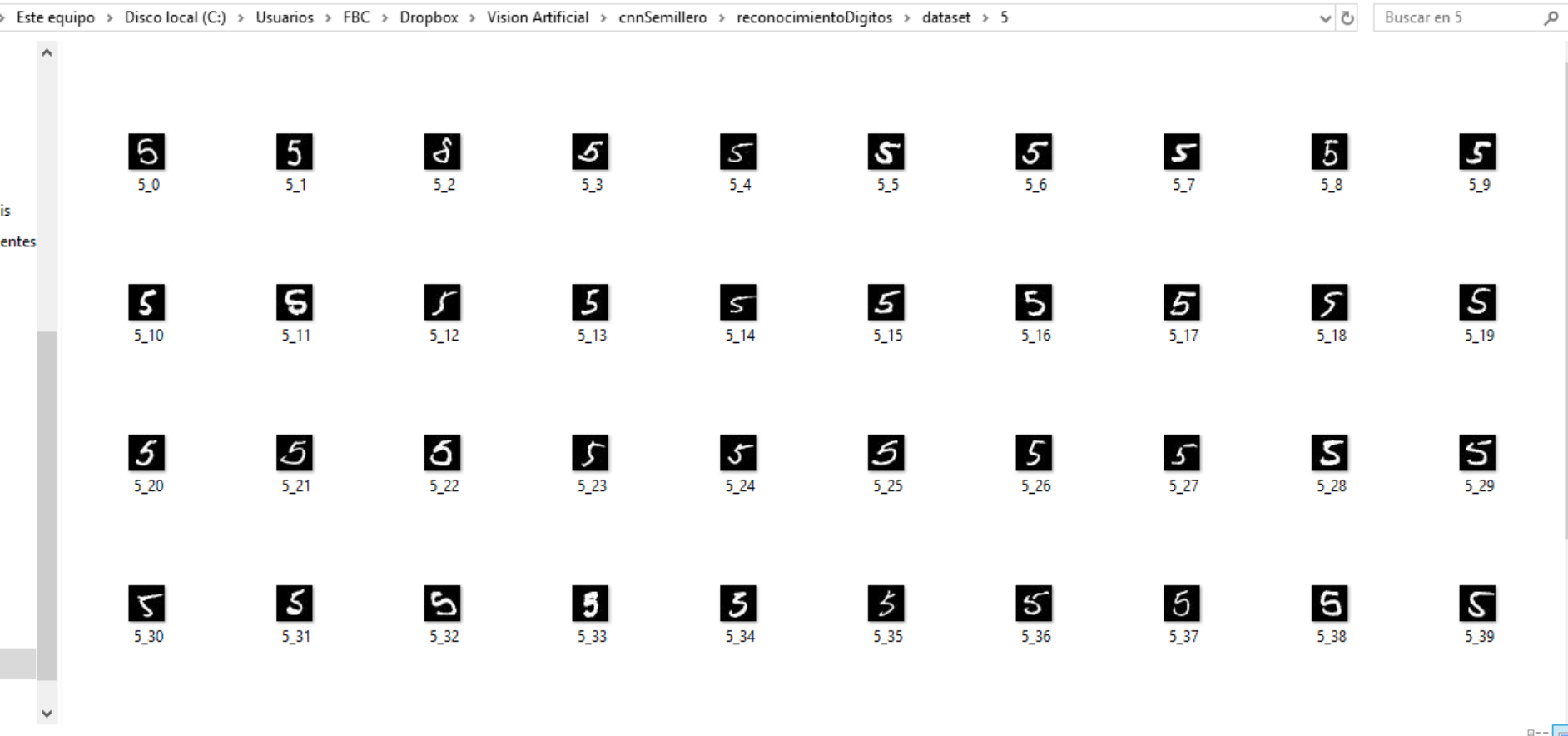
Datos prueba

Dataset



Datos prueba

Dataset



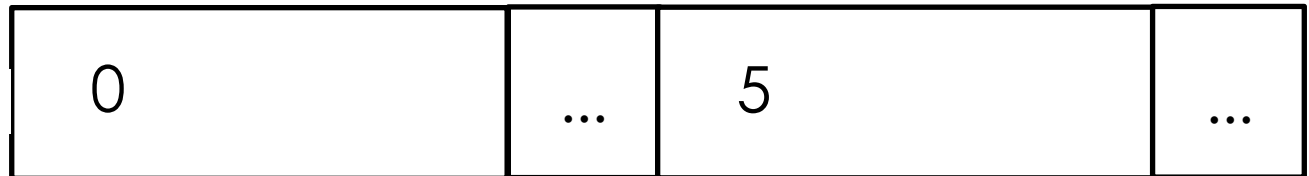
Datos prueba

Detrás aprendizaje supervisado

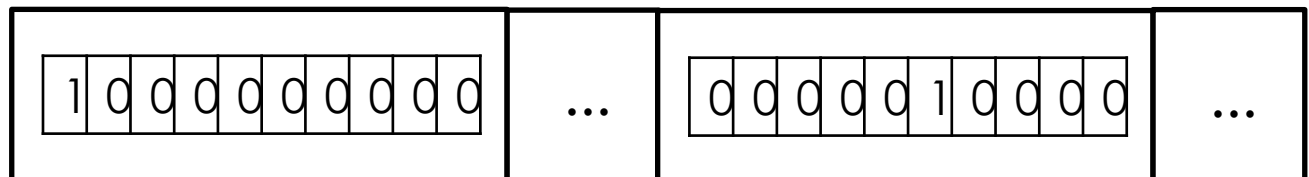
imagenesEntrenamiento



etiquetasEntrenamiento

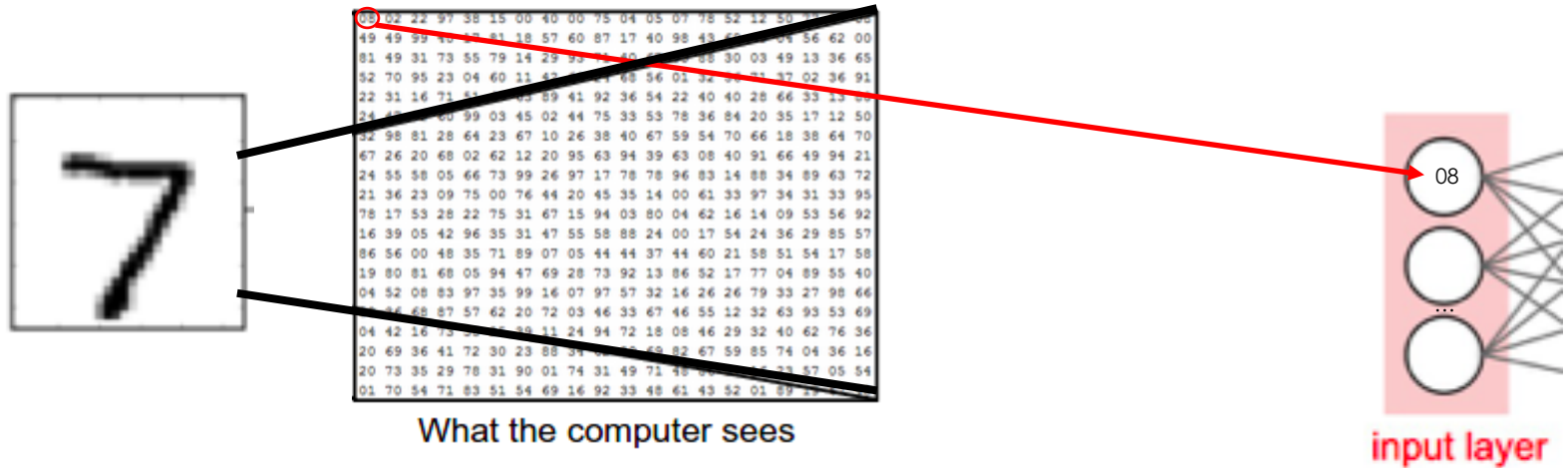


valoresEsperados



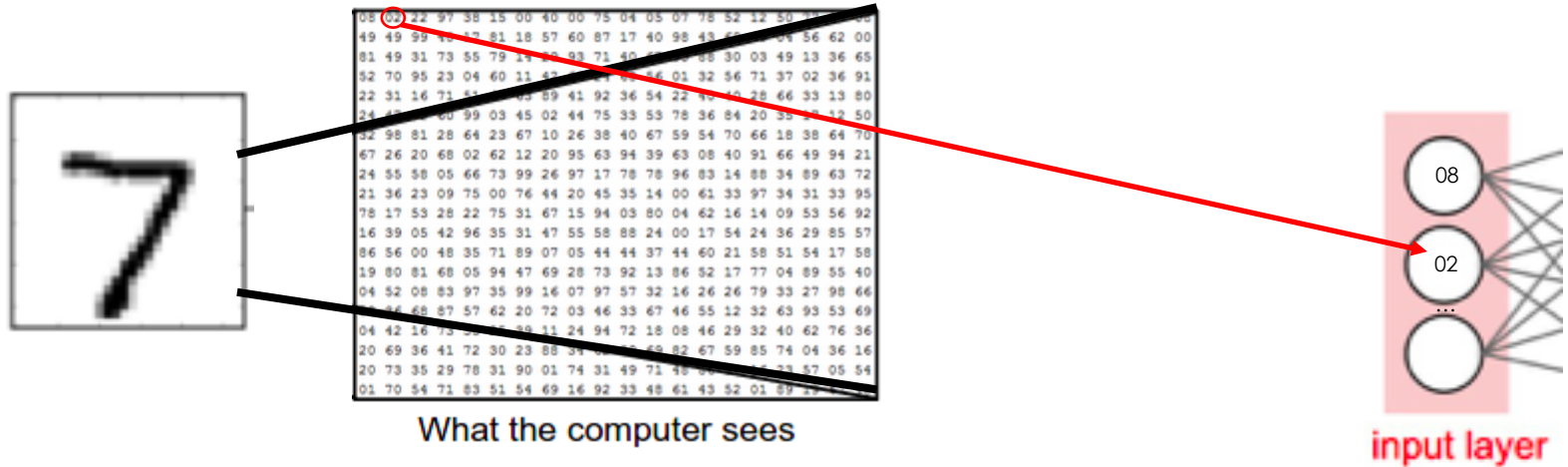
Configuración

Detrás aprendizaje supervisado



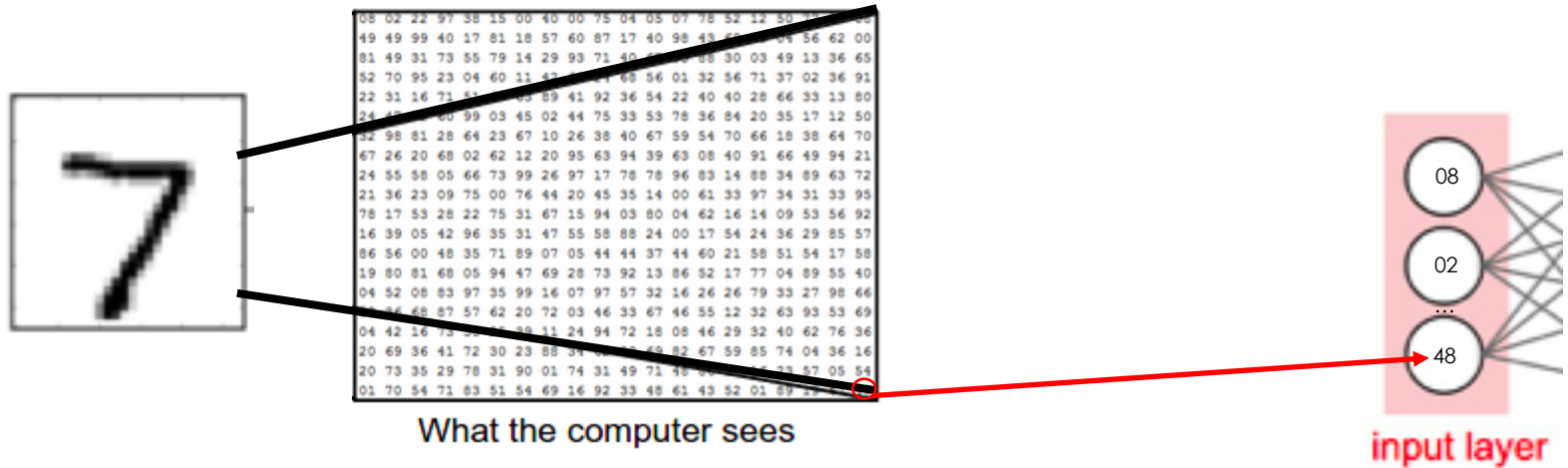
Capa de entrada de la red

Detrás aprendizaje supervisado



Capa de entrada de la red

Detrás aprendizaje supervisado



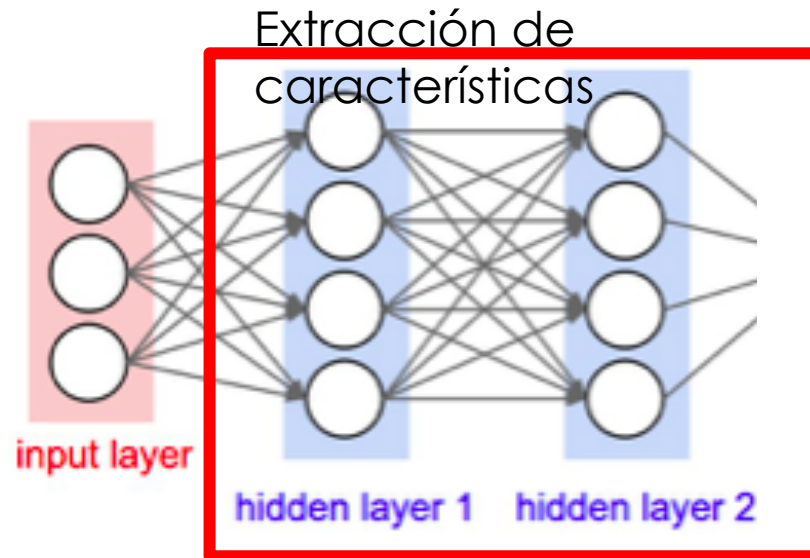
Capa de entrada de la red

Código en Python

```
model=Sequential()  
#Capa entrada  
model.add(InputLayer(input_shape=(img_size_flat,)))
```

Capa inicial entrada red

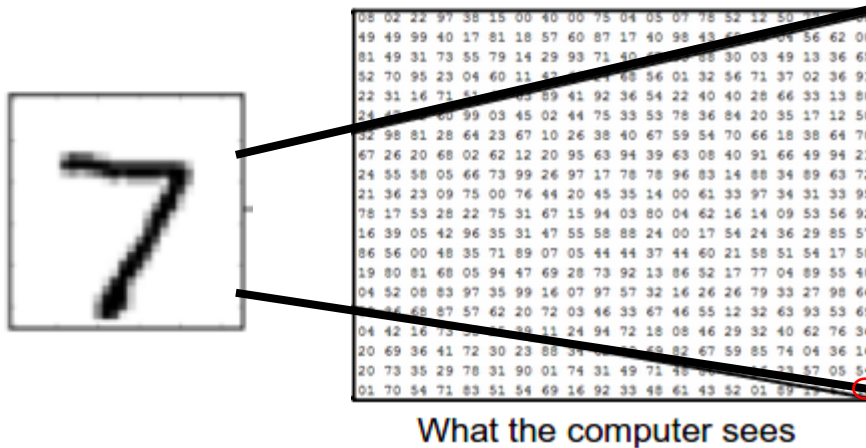
Estructuras de las capas



Operaciones: Convoluciones, Pooling,
Funciones de activación

Google Developers, <https://developers.google.com/machine-learning/practica/image-classification/convolutional-neural-networks>

Detrás aprendizaje supervisado



Capa de entrada de la red

Convoluciones 2D

3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

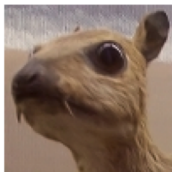
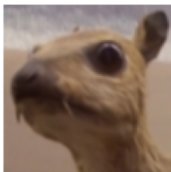
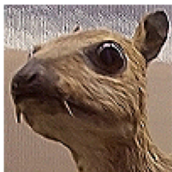
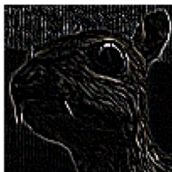
https://miro.medium.com/max/1400/1*Fw-ehcNBR9byHtho-Rxbtw.gif

Kernel se "desliza" sobre los datos de entrada 2D, realizando una multiplicación por elementos con la parte de la entrada en la que se encuentra en ese momento, y luego sumando los resultados en un único píxel de salida.

Convolución y Pooling

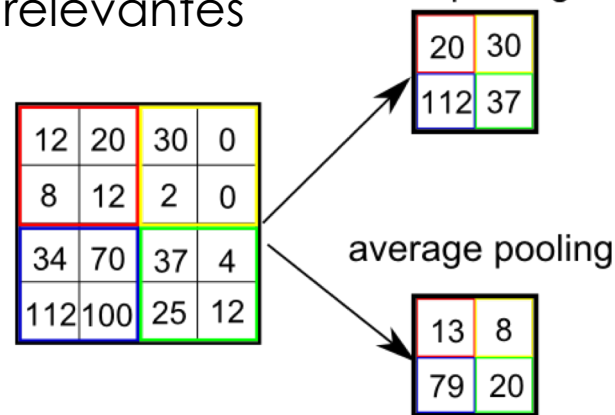
Convolución

Pasar filtros sobre la imagen y realzar detalles (Puede

Original	Gaussian Blur	Sharpen	Edge Detection
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
			

Pooling

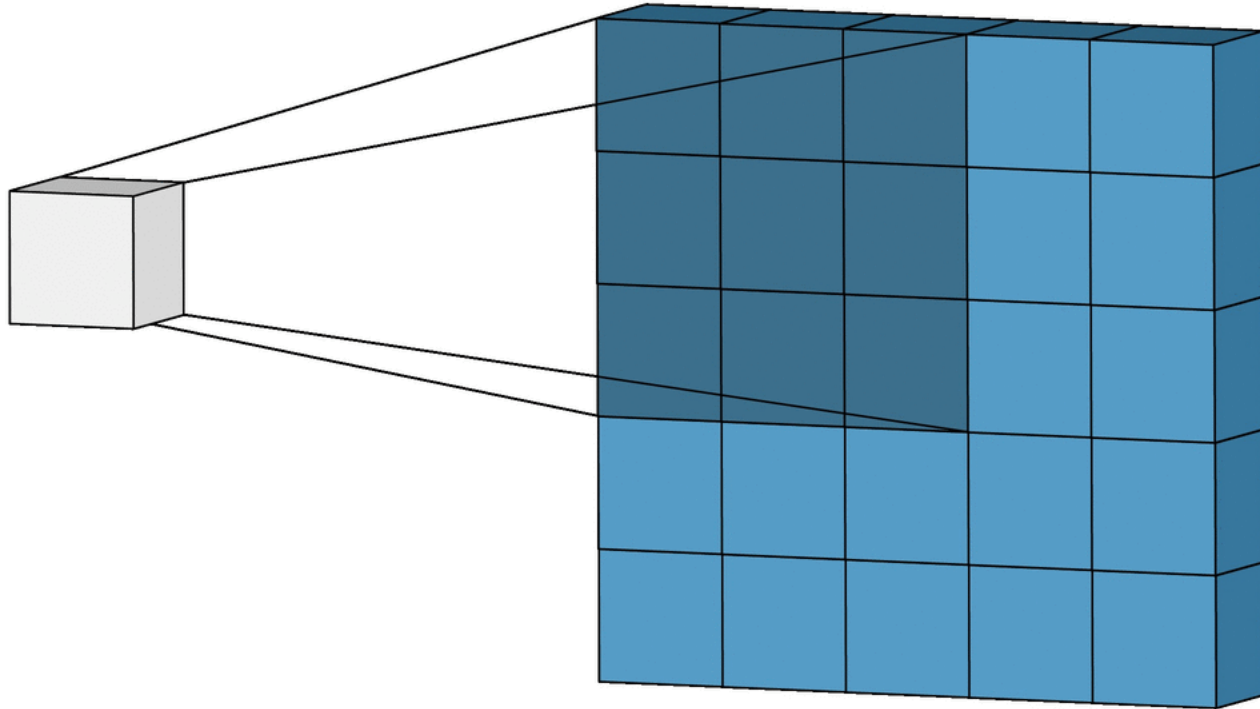
Reducción de las imágenes obteniendo características mas relevantes



```
# Primera capa convolucional con ReLU-activation y max-pooling.
model.add(Conv2D(kernel_size=5, strides=1, filters=16, padding='same', activation='relu', name='layer_conv1'))
model.add(MaxPooling2D(pool_size=2, strides=2))

# Segunda capa convolucional con ReLU-activation y max-pooling.
#padding = same --> imagen de salida mismo tamaño de la entrada
model.add(Conv2D(kernel_size=5, strides=1, filters=36, padding='same', activation='relu', name='layer_conv2'))
model.add(MaxPooling2D(pool_size=2, strides=2))
```

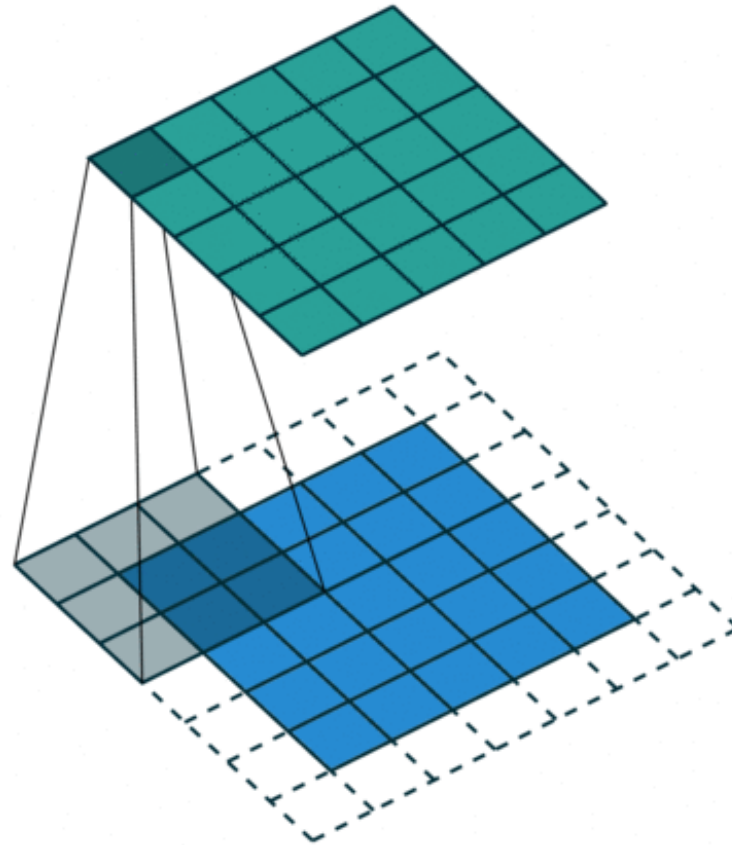
Pooling



https://miro.medium.com/max/1400/1*Fw-ehcNBR9byHtho-Rxbtw.gif

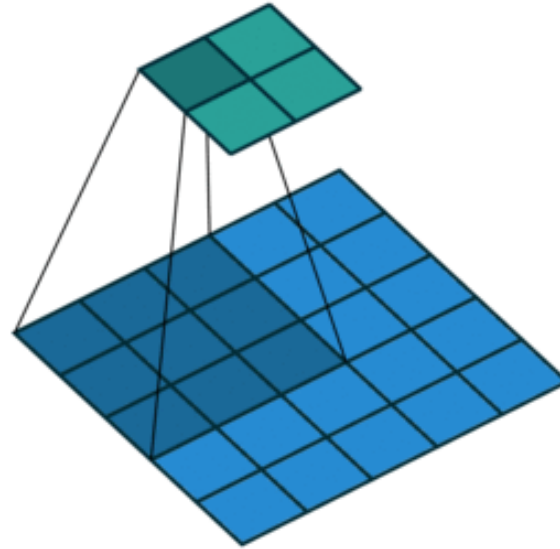
Desliza filtros sobre la imagen original para realzar características

Padding -- Relleno



Durante el proceso de deslizamiento, los bordes se "recortan" esencialmente, convirtiendo una matriz de características de 5×5 en una de 3×3 . Rellenar los bordes con píxeles extra, "falsos" (normalmente de valor 0, de ahí el término tan usado de "relleno cero"). zero padding"

Striding -- Desplazamiento



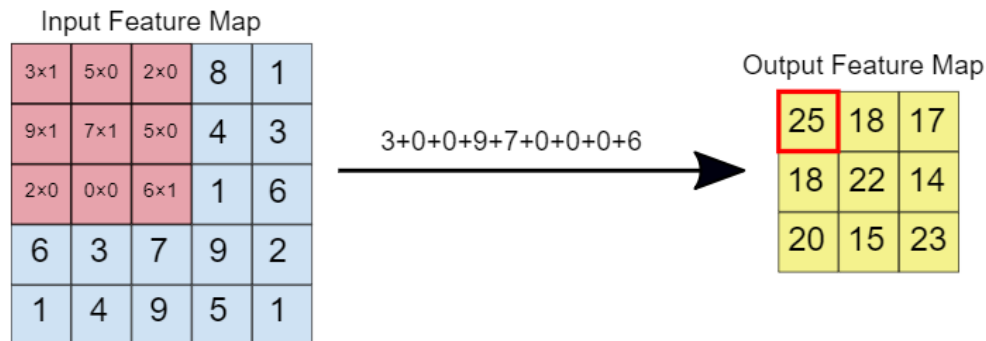
Permite reducir el tamaño de la entrada, permite saltarse de acuerdo con un parámetro el recorrido de la matriz de entrada. Ejemplo. Un intervalo de 2 significa escoger "sl" con 2 píxeles de diferencia, saltando cada "slide" en el proceso, reduciendo el tamaño en un factor de 2,

Tamaño salida Pooling

$$w_f = \frac{w_i - f}{s} + 1$$

$$h_f = \frac{h_i - f}{s} + 1$$

wf=Ancho final
hf=Altura final
f= Ancho o Altura
del filtro
s= stride



Desliza filtros sobre la imagen original para realzar características

Estructuras de las capas ocultas - convolución

Desliza filtros sobre la imagen original para realzar características

Input Feature Map

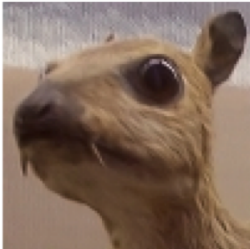
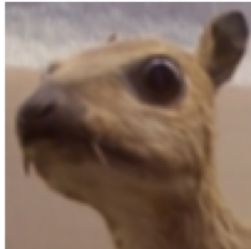
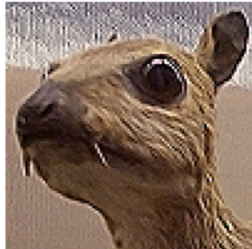
3	5	2	8	1
9	7	5	4	3
2	0	6	1	6
6	3	7	9	2
1	4	9	5	1

Convolutional Filter

1	0	0
1	1	0
0	0	1

Google Developers, <https://developers.google.com/machine-learning/practica/image-classification/convolutional-neural-networks>

Estructuras de las capas ocultas - convolución

<i>Original</i>	<i>Gaussian Blur</i>	<i>Sharpen</i>	<i>Edge Detection</i>
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
			

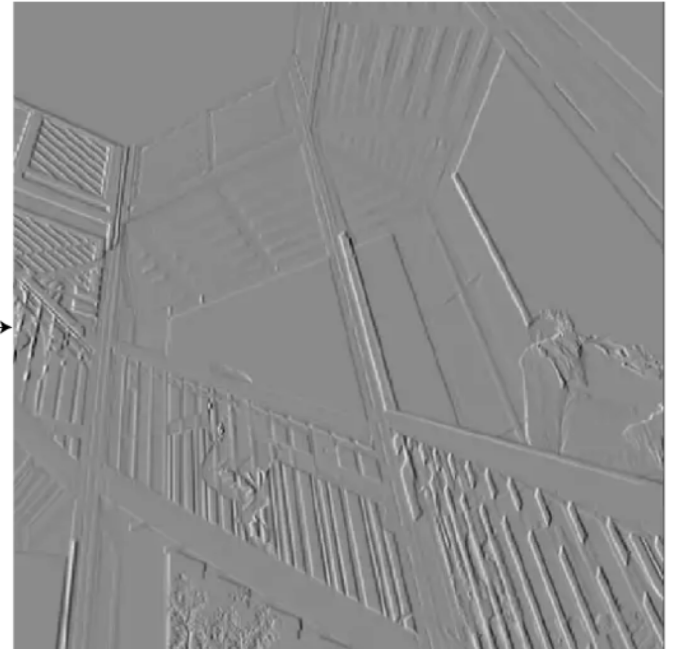
Desliza filtros sobre la imagen original para realzar características

Estructuras de las capas ocultas - convolución



$$\begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix}$$

Horizontal Sobel kernel



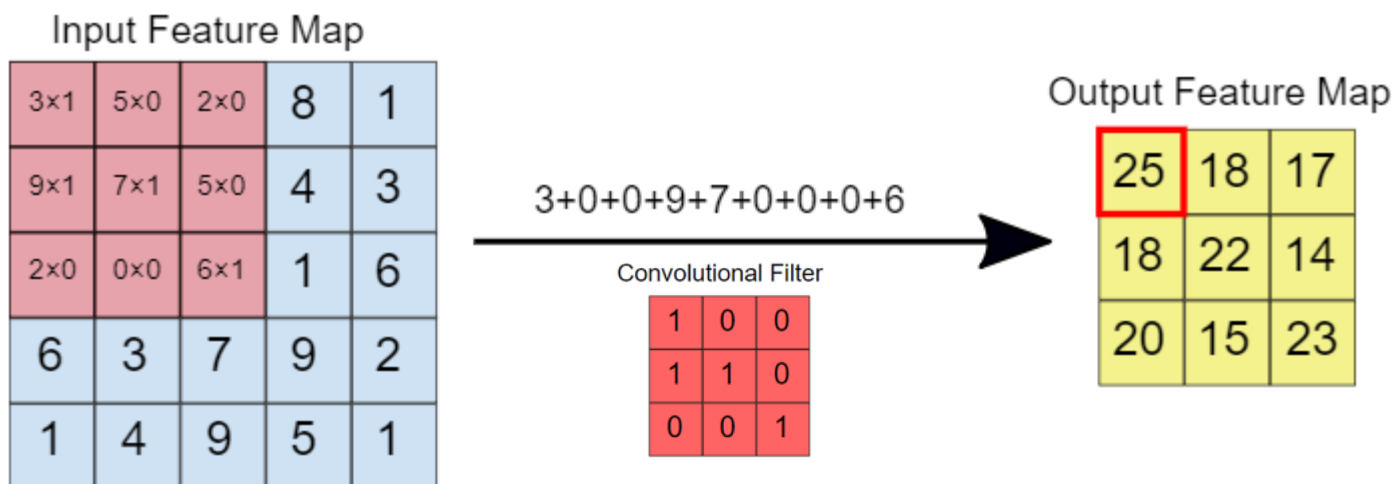
Applying a vertical edge detector kernel

https://miro.medium.com/max/1400/1*wju0Urp6KpAT11wktTp5Sg.webp

Desliza filtros sobre la imagen original para realzar características

Estructuras de las capas ocultas - convolución

Desliza filtros sobre la imagen original para realzar características

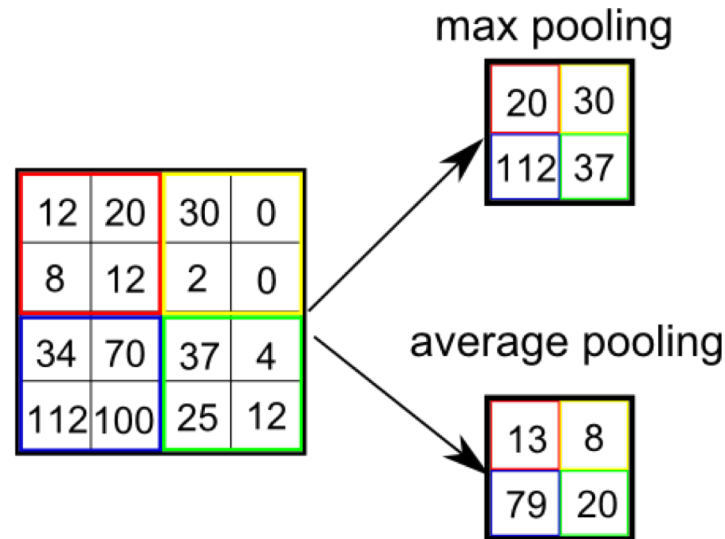


Google Developers, <https://developers.google.com/machine-learning/practica/image-classification/convolutional-neural-networks>

Desliza filtros sobre la imagen original para realzar características

Pooling

Reduce la matriz convolucionada para la siguiente capa

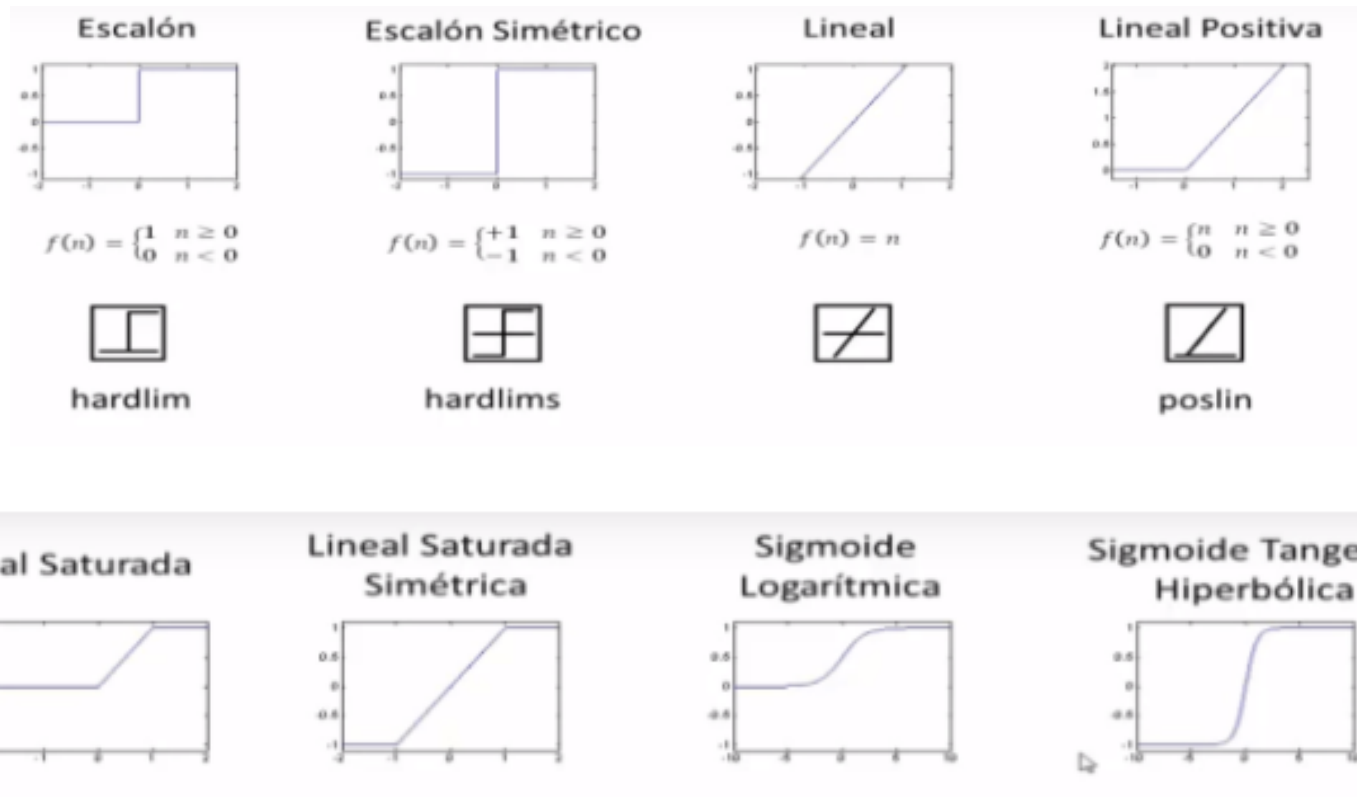


Google Developers, <https://developers.google.com/machine-learning/practica/image-classification/convolutional-neural-networks>

Desliza filtros sobre la imagen original para realzar características

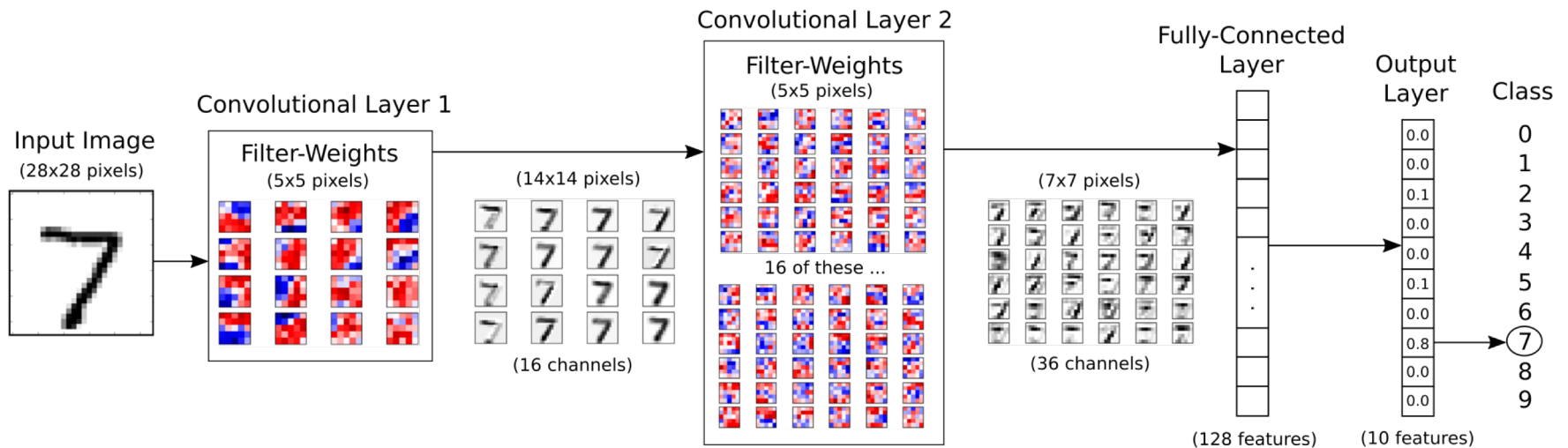
Funciones activación

Tiene como fin introducir la no linealidad en el modelo



Ver presentación de funciones de activación

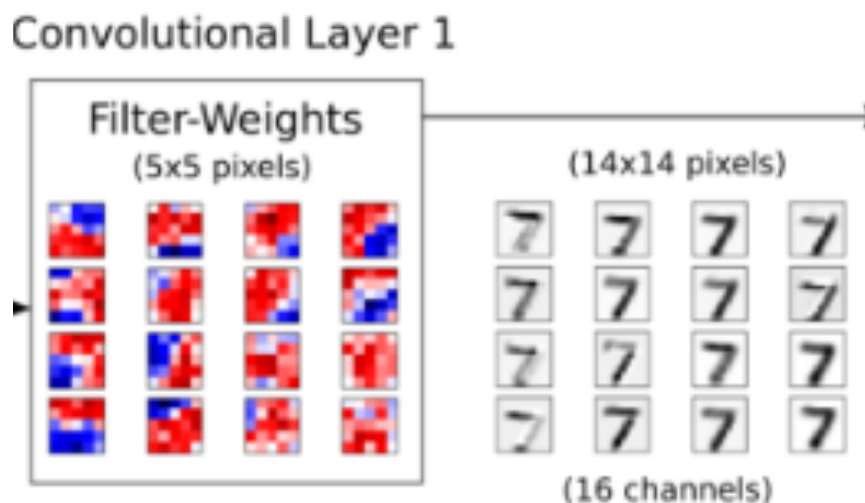
Estructuras de las capas ocultas - convolución



Pipeline – Flujo procesamiento desde la entrada a la salida

Estructuras de las capas ocultas - convolución

Desliza filtros sobre la imagen original para realzar características

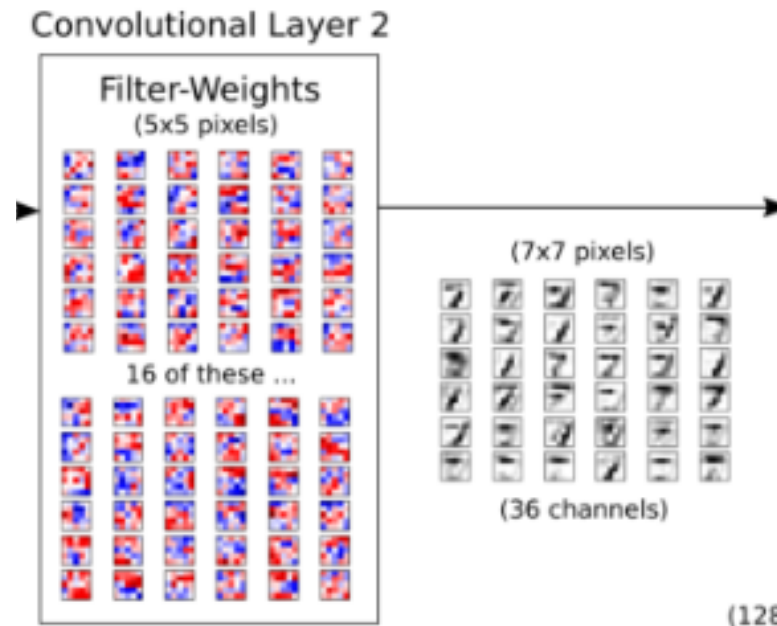


Apsl, <https://www.apsl.net/blog/2018/02/02/tensor-flow-para-principiantes-vi-uso-de-la-api-keras/>

Pipeline – Flujo procesamiento desde la entrada a la salida

Estructuras de las capas ocultas - convolución

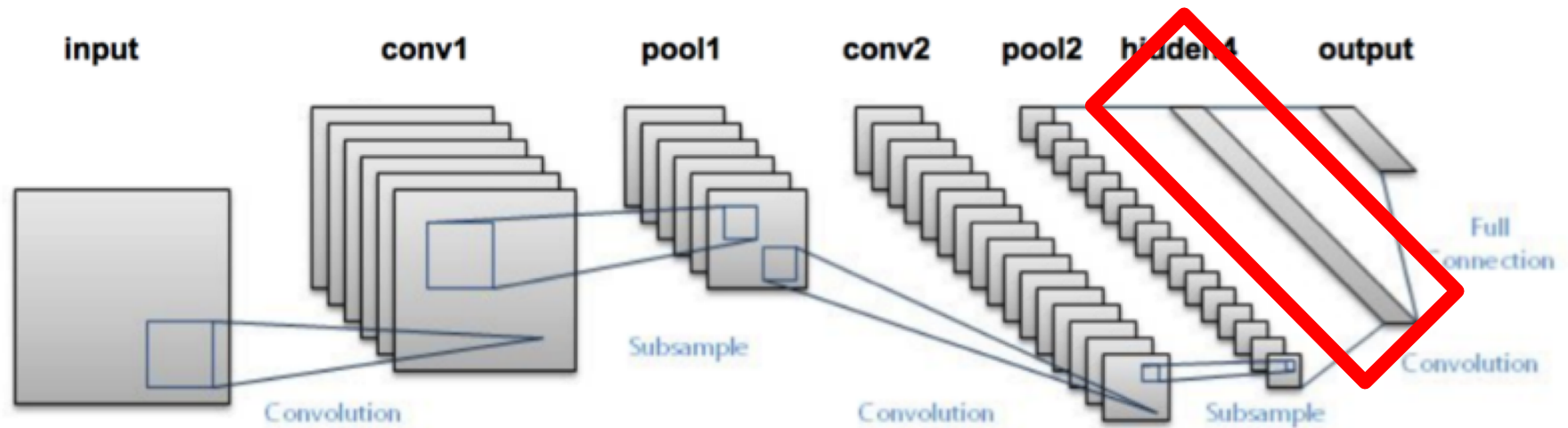
Desliza filtros sobre la imagen original para realzar características



[https://www.apsl.net/blog/2018/02/02/tensor-flow-para-principiantes-vi-
uso-de-la-api-keras/](https://www.apsl.net/blog/2018/02/02/tensor-flow-para-principiantes-vi-
uso-de-la-api-keras/)

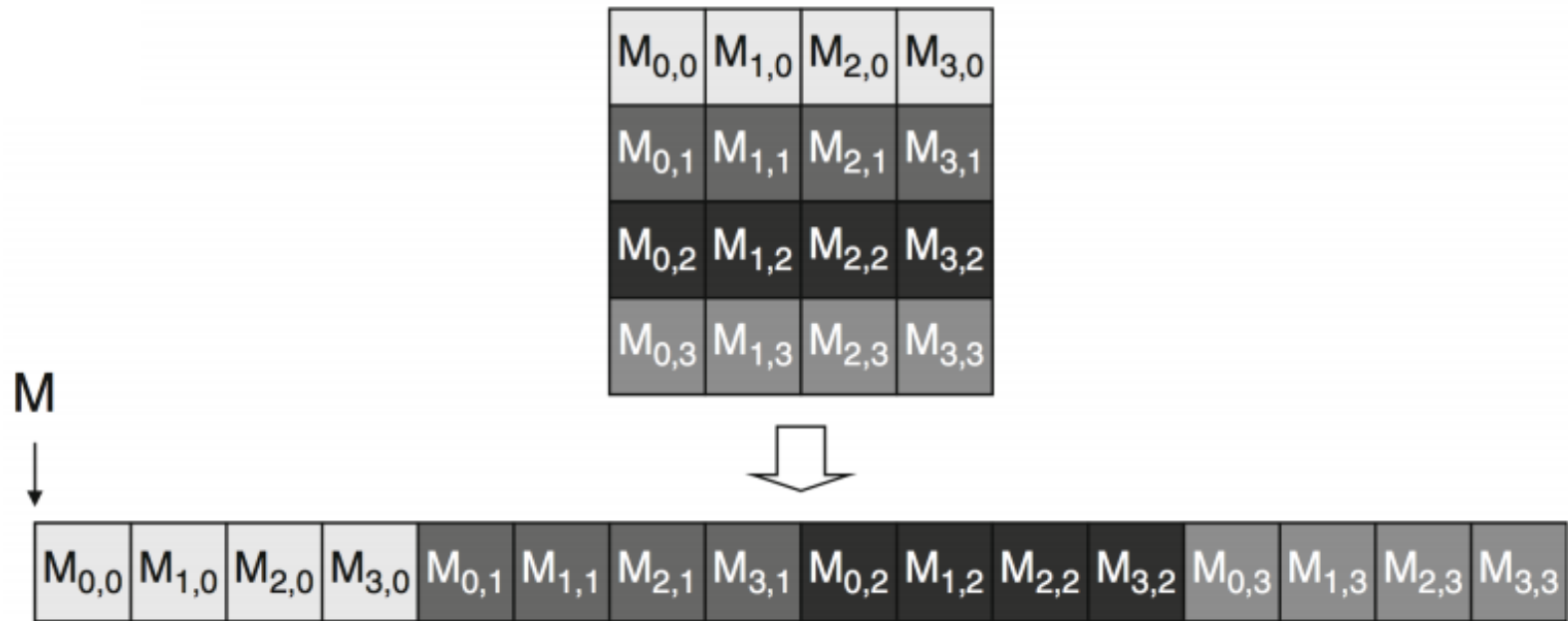
Pipeline – Flujo procesamiento desde la entrada a la salida

Estructuras de las capas ocultas - convolución



Pipeline – Flujo procesamiento desde la entrada a la salida

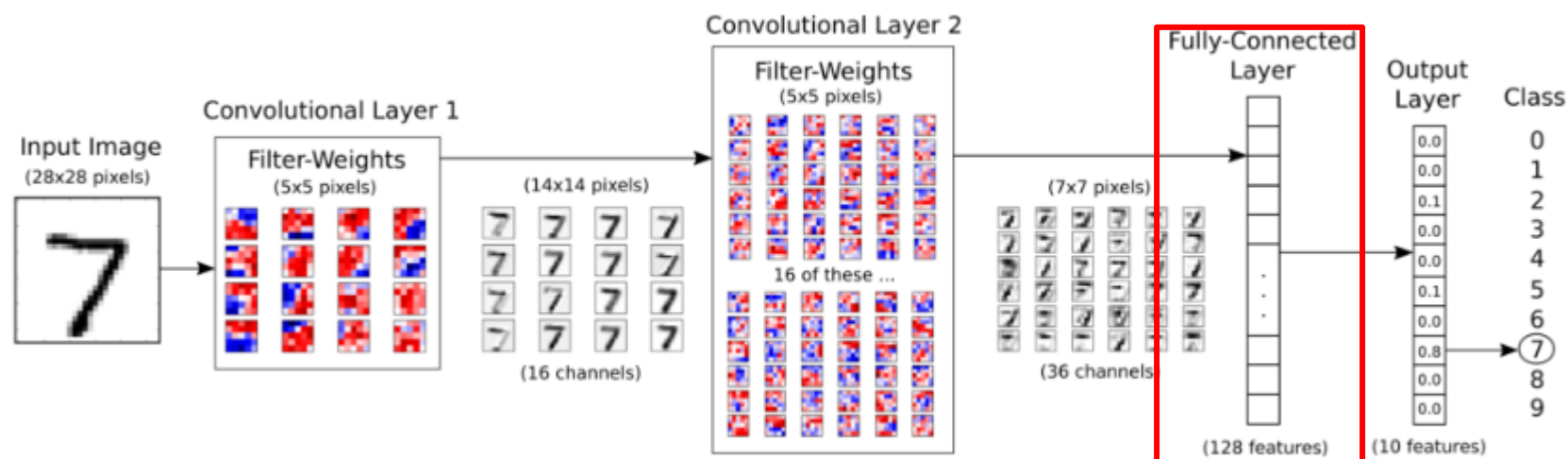
Estructuras de las capas ocultas - convolución



Aplanamiento --- Flatten

Estructuras de las capas ocultas – directamente conectadas

Realiza la clasificación basado en las características obtenidas de las convoluciones. Estas por lo general tienen una función de activación softmax la cual brinda un valor de probabilidad de 0 a 1 por cada una de las salidas



, <https://www.apsl.net/blog/2018/02/02/tensor-flow-para-principiantes-vi-uso-de-la-api-keras/>

Aplanamiento --- Flatten

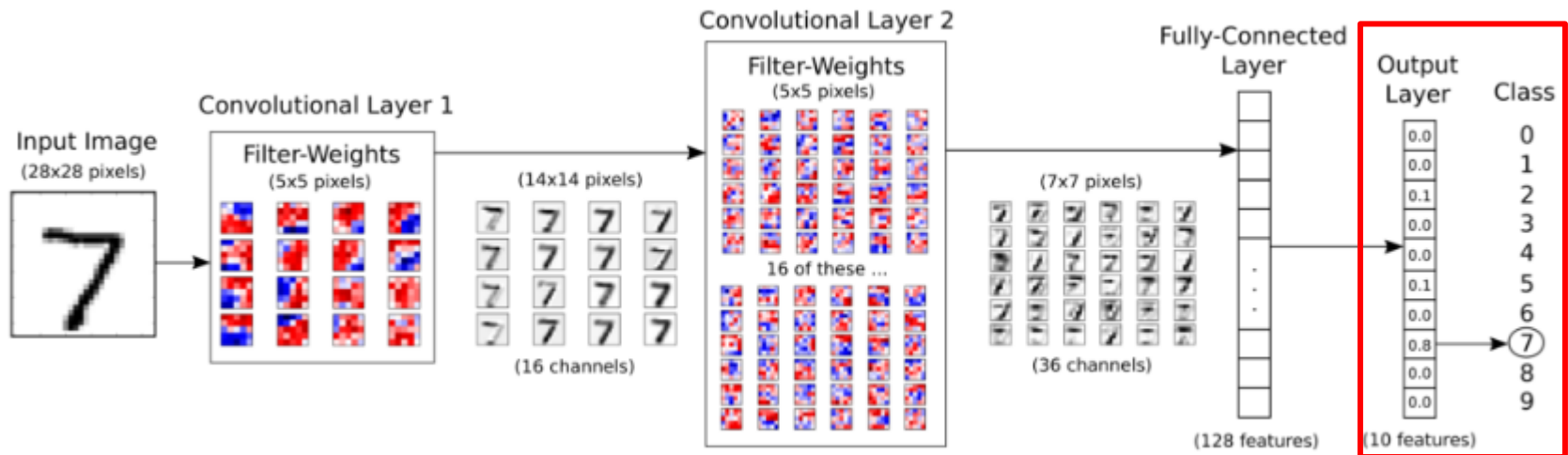
Codificación en Python

```
#Aplanar imagen  
model.add(Flatten())  
#Capa densa  
model.add(Dense(128, activation='relu'))
```

Aplanamiento --- Flatten

Estructuras de la capa de salida

Según el número de etiquetas de salida, proporciona un valor entre 0 y 1. Aquella que posea el valor mas alto, será la clasificación final



61

<https://www.apsl.net/blog/2018/02/02/tensor-flow-para-principiantes-vi-uso-de-la-api-keras/>

Pipeline – Flujo procesamiento desde la entrada a la salida

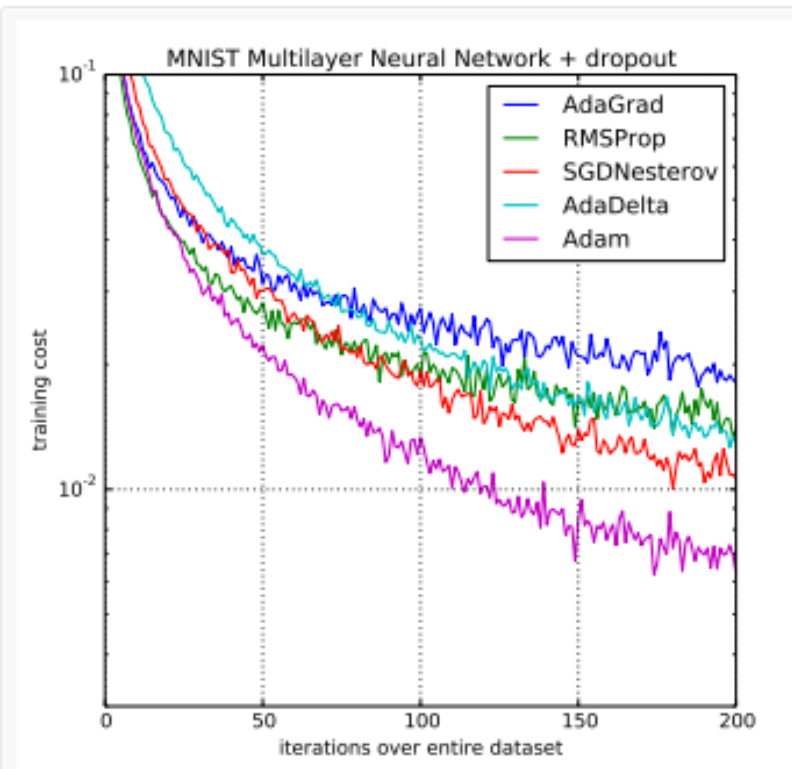
Codificación en Python

```
#Capa salida  
model.add(Dense(num_clases,activation='softmax'))
```

Capa de Salida

Entrenamiento de la Red Neuronal Convolución

Control Estocástico



Comparación de Adam con otros algoritmos de optimización que forman un perceptrón multicapa
tomado de Adam: un método para la optimización estocástica, 2015.

Codificación en Python

```
#Compilacioon del modelo  
optimizador=Adam(lr=1e-3)  
model.compile(optimizer=optimizador,  
               loss='categorical_crossentropy',  
               metrics=['accuracy'])
```

Compilación del modelo

Codificación en Python

```
#Entrenamiento del modelo
```

```
model.fit(x=imagenes,y=probabilidades,epochs=1,batch_size=100)
```

```
limiteImagenesPrueba=20
```

```
imagenesPrueba,etiquetasPrueba,probabilidadesPrueba=cargarDatos("test/",num_clases,limiteImagenesPrueba
```

```
resultados=model.evaluate(x=imagenesPrueba,y=probabilidadesPrueba)
```

```
print("{0}: {1:.2%}".format(model.metrics_names[1], resultados[1]))
```

```
nombreArchivo='modeloReconocimientoNumerosNuevo.keras'
```

```
model.save(nombreArchivo)
```

```
model.summary()
```

Entrenamiento y guardar
el modelo (pesos)

Pruebas

Codificación en Python

```
from tensorflow.python.keras.models import load_model
import numpy as np
import cv2

class prediccion():
    def __init__(self):
        self.rutaModelo="models/modeloReconocimientoNumeros.keras"
        self.model=load_model(self.rutaModelo)
        self.width=28
        self.height=28
    def predecir(self, imagen):
        imagen=cv2.resize(imagen, (self.width, self.height))
        imagen=imagen.flatten()
        imagen=np.array(imagen)
        imagenNormalizada=imagen/255
        pruebas=[]
        pruebas.append(imagenNormalizada)
        imagenesAPredecir=np.array(pruebas)
        predicciones=self.model.predict(x=imagenesAPredecir)
        claseMayorValor=np.argmax(predicciones, axis=1)
        return claseMayorValor[0]
```

Usar la red para predecir

Codificación en Python

```
import cv2

from cnnSemillero.reconocimientoDigitos.prediccion import prediccion

categorias=["0","1","2","3","4","5","6","7","8","9"]
reconocimiento=prediccion()
imagenPrueba=cv2.imread("test/8_0.jpg",0)
indiceCategoria=reconocimiento.predecir(imagenPrueba)
print("La imagen cargada es ",categorias[indiceCategoria])
while True:
    cv2.imshow("imagen",imagenPrueba)
    k=cv2.waitKey(30) & 0xff
    if k==27:
        break
cv2.destroyAllWindows()
```

Usar la red para predecir

Codificación en Python

```
}import cv2

}from cnnSemillero.reconocimientoDigitos.prediccion import prediccion

categorias=["0","1","2","3","4","5","6","7","8","9"]
reconocimiento=prediccion()
imagenPrueba=cv2.imread("test/8_0.jpg",0)
indiceCategoria=reconocimiento.predecir(imagenPrueba)
print("La imagen cargada es ",categorias[indiceCategoria])
}while True:
    cv2.imshow("imagen",imagenPrueba)
    k=cv2.waitKey(30) & 0xff
    if k==27:
        break
}
cv2.destroyAllWindows()
```

Codificación Base 64

Link Github

<https://github.com/luisfercastilloCOL/CNNforImages>

Link Jupiter Notebook

Reconocimiento imágenes Python



Sistemas Inteligentes 2

Luis Fernando Castillo